Review 4

Professor Amanda Bienz

Conditional Variables

- **pthread_cond_wait**
- **pthread_cond_signal**
- **pthread_cond_broadcast**
- *When do you use each of the above?*
- *What is the difference between signal and broadcast?*

Parent Waiting on Child

```
1       void thr_exit() {
2               Pthread_mutex_lock(&m);
3               Pthread_cond_signal(&c);
4               Pthread_mutex_unlock(&m);
5       }
6
7       void thr_join() {
8               Pthread_mutex_lock(&m);
9               Pthread_cond_wait(&c, &m);
10              Pthread_mutex_unlock(&m);
11      }
```

- What could go wrong here?

Producer / Consumer Problem

- Step through slides (Concurrency-Conditions slide deck) and note what can go wrong with different scenarios
  - What if pthread_cond_wait(…) is not inside a while loop?
  - What if both producer and consumer signal and wait on same condition variable?

Allocating Data

- How do we use condition variables to wake up a specific thread?
- i.e. zero bytes are currently free. T0 wants to allocate 100 bytes, T1 wants to allocate 10 bytes, and T2 frees 50 bytes. How do you guarantee that T1 wakes up?
  - What goes wrong if T2 calls pthread_cond_signal?
  - What should be called?

Producer / Consumer and Semaphores

- Again, step through slides (Concurrency Alternatives slide deck) and make sure you understand what goes wrong in different examples
  - What if there is no lock around put() or get()?
  - What if the lock is before the semaphore?
  - Where should the lock be?
  - What should the different semaphores be initialized to?

Reader-Writer Locks

- Assume writer threads want to write to a buffer (or change a list) and reader threads want to read from a buffer (or read a list)
  - Do all threads need to be locked?
  - Can any two threads operate at the same time?
  - What are some problems you can run into with this type of lock?

Dining Philosophers

- Again, go through slides (Concurrency Bugs slide deck) and make sure you understand what is incorrect with different examples, and also understand the different strategies for fixing this deadlock.
  - All philosophers grab left fork and then right fork?
  - Is it fixed if only one philosopher grabs right before left? Why or why not?
  - What is necessary for a deadlock to occur?
    - Mutual exclusion, hold-and-wait, no preemption, circular wait